

Neo4j CQL – Introduction

CQL stands for Cypher Query Language. Like Oracle Database has query language SQL, Neo4j has CQL as query language.

Neo4j CQL

- Is a query language for Neo4j Graph Database.
- Is a declarative pattern-matching language.
- Follows SQL like syntax.
- Syntax is very simple and in human readable format.

Neo4j CQL Clauses

Following are the read clauses of Neo4j Cypher Query Language –

Sr.No	Read Clauses	Usage
1	MATCH	This clause is used to search the data with a specified pattern.
2	OPTIONAL MATCH	This is the same as match, the only difference being it can use nulls in case of missing parts of the pattern.
3	WHERE	This clause id is used to add contents to the CQL queries.
4	START	This clause is used to find the starting points through the legacy indexes.
5	LOAD CSV	This clause is used to import data from CSV files.

Following are the write clauses of Neo4j Cypher Query Language –

Sr.No	Write Clause	Usage
1	CREATE	This clause is used to create nodes, relationships, and properties.
2	MERGE	This clause verifies whether the specified pattern exists in the graph. If not, it creates the pattern.
3	SET	This clause is used to update labels on nodes, properties on nodes and relationships.
4	DELETE	This clause is used to delete nodes and relationships or paths etc. from the graph.
5	REMOVE	This clause is used to remove properties and elements from nodes and relationships.
6	FOREACH	This class is used to update the data within a list.
7	CREATE UNIQUE	Using the clauses CREATE and MATCH, you can get a unique pattern by matching the existing pattern and creating the missing one.
8	Importing CSV files with Cypher	Using Load CSV you can import data from .csv files.

Following are the general clauses of Neo4j Cypher Query Language –

Sr.No	General Clauses	Usage
1	RETURN	This clause is used to define what to include in the query result set.
2	ORDER BY	This clause is used to arrange the output of a query in order. It is used along with the clauses RETURN or WITH .
3	LIMIT	This clause is used to limit the rows in the result to a specific value.
4	SKIP	This clause is used to define from which row to start including the rows in the output.
5	WITH	This clause is used to chain the query parts together.
6	UNWIND	This clause is used to expand a list into a sequence of rows.
7	UNION	This clause is used to combine the result of multiple queries.
8	CALL	This clause is used to invoke a procedure deployed in the database.

Neo4j CQL Functions

Following are the frequently used Neo4j CQL Functions –

Sr.No	CQL Functions	Usage
1	String	They are used to work with String literals.
2	Aggregation	They are used to perform some aggregation operations on CQL Query results.
3	Relationship	They are used to get details of relationships such as startnode, endnode, etc.

CQL Operators

Following are the list of operators supported by Neo4j Cypher Query language.

Sr.No	Type	Operators
1	Mathematical	+, -, *, /, %, ^
2	Comparison	+, <>, <, >, <=, >=
3	Boolean	AND, OR, XOR, NOT
4	String	+
5	List	+, IN, [X], [X..Y]
6	Regular Expression	=~
7	String matching	STARTS WITH, ENDS WITH, CONSTRAINTS

Neo4j supports the following Boolean operators to use in Neo4j CQL WHERE clause to support multiple conditions.

Sr.No	Boolean Operators	Description
1	AND	It is a Neo4j CQL keyword to support AND operation. It is like SQL AND operator.
2	OR	It is a Neo4j CQL keyword to support OR operation. It is like SQL AND operator.
3	NOT	It is a Neo4j CQL keyword to support NOT operation. It is like SQL AND operator.
4	XOR	It is a Neo4j CQL keyword to support XOR operation. It is like SQL AND operator.

Comparison Operators in Neo4j CQL

Neo4j supports the following Comparison operators to use in Neo4j CQL WHERE clause to support conditions.

Sr.No	Boolean Operators	Description
1	=	It is a Neo4j CQL "Equal To" operator.
2	< >	It is a Neo4j CQL "Not Equal To" operator.
3	<	It is a Neo4j CQL "Less Than" operator.
4	>	It is a Neo4j CQL "Greater Than" operator.
5	<=	It is a Neo4j CQL "Less Than Or Equal To" operator.
6	> =	It is a Neo4j CQL "Greater Than Or Equal To" operator.

Neo4j CQL – Creating Nodes

Creating a Single Node: Following is a syntax for creating new node

Syntax,

```
CREATE (node_name);
```

Example,

```
CREATE (babar:player)
```

Verification: Execute the following query to verify the creation of new node.

```
MATCH (n) RETURN n
```

Creating a Multiple Nodes: The create clause of Neo4j CQL is also used to create multiple nodes at the same time. To do so, you need to pass the names of the nodes to be created, separated by a comma.

Syntax,

```
CREATE (node1),(node2)
```

Example,

```
CREATE (sample1),(sample2)
```

Creating a Node with a Label: A label in Neo4j is used to group (classify) the nodes using labels. You can create a label for a node in Neo4j using the CREATE clause.

Syntax,

```
CREATE (node:label)
```

Example,

```
CREATE (babar:player)
```

Creating a Node with Properties: Properties are the key-value pairs using which a node stores data. You can create a node with properties using the CREATE clause. You need to specify these properties separated by commas within the flower braces { }.

Syntax,

```
CREATE (node:label { key1: value, key2: value, . . . . . })
```

Example,

```
CREATE (babar:player{name: "babar azam", YOB:1990, POB: "lahore"})
```

Creating a Node with Return: Throughout the chapter, we used the MATCH (n) RETURN n query to view the created nodes. This query returns all the existing nodes in the database. Instead of this, we can use the RETURN clause with CREATE to view the newly created node.

Syntax,

```
CREATE (Node:Label{properties. . . }) RETURN Node
```

Example,

```
CREATE (babar:player{name: "babar azam", YOB: 1990, POB: "lahore"}) RETURN babar
```

Neo4j CQL – Creating a Relationship

We can create a relationship using the CREATE clause. We will specify relationship within the square braces [] depending on the direction of the relationship it is placed between hyphen - and arrow → as shown in the following syntax.

Syntax,

```
CREATE (node1)-[:RelationshipType]->(node2)
```

Example,

```
CREATE(babar:player{name: "babar azam", YOB:1990,POB:"lahore"}) CREATE(pak:Country {name:"pak"})
```

Now, create a relationship named **BATSMAN_OF** between these two nodes as –

```
CREATE (babar)-[r:BATSMAN_OF]->(pak)
```

Creating a relationship between the existing nodes: You can also create a relationship between the existing nodes using the **MATCH** clause.

Syntax,

```
MATCH (a:LabeofNode1), (b:LabeofNode2)
```

```
WHERE a.name = "nameofnode1" AND b.name = " nameofnode2"
```

```
CREATE (a)-[: Relation]->(b)
```

```
RETURN a,b
```

Example,

```
MATCH (a:player), (b:Country) WHERE a.name = "babar azam" AND b.name = "pak" CREATE (a)-[r: BATSMAN_OF]->(b) RETURN a,b
```

Creating a relationship with Label and Properties: You can create a relationship with label and properties using the CREATE clause.

Following is the syntax to create a relationship with label and properties using the CREATE clause.

```
CREATE (node1)-[label:Rel_Type {key1:value1, key2:value2, . . . n}]-> (node2)
```

Example,

```
MATCH(a:player), (b:Country) WHERE a.name = "babar azam" AND b.name = "pak" CREATE(a)-[r:BATSMAN_OF {Matches:5, Avg:90.75}]->(b) RETURN a,b
```

Creating a complete Path: In Neo4j, a path is formed using continuous relationships. A path can be created using the create clause.

Syntax,

```
CREATE p = (Node1 {properties})-[:Relationship_Type]->
  (Node2 {properties})[:Relationship_Type]->(Node3 {properties})
RETURN p
```

Neo4j CQL – Merge Command

MERGE command is a combination of CREATE command and MATCH command.

Neo4j CQL MERGE command searches for a given pattern in the graph. If it exists, then it returns the results.

If it does NOT exist in the graph, then it creates a new node/relationship and returns the results.

Syntax,

```
MERGE (node: label {properties . . . . . })
```

Before proceeding to the examples in this section, create two nodes in the database with labels babar and pak. Create a relationship of type BATSMAN_OF from babar to pak as shown below.

```
CREATE (babar:player{name: "babar azam", YOB: 1990, POB: "lahore"}) CREATE (pak:Coun  
try {name: "pakistan"}) CREATE (babar)-[r:BATSMAN_OF]->(pak)
```

Match a node with Label: You can merge a node in the database based on the label using the MERGE clause. If you try to merge a node based on the label, then Neo4j verifies whether there exists any node with the given label. If not, the current node will be created.

Syntax,

```
MERGE (node:label) RETURN node
```

Example,

```
MERGE (afridi:player) RETURN afridi
```

```
MERGE (afridi:player {name: "shaheen afridi", YOB: 1998, POB: "peshawar"})  
RETURN afridi
```

OnCreate and OnMatch: Whenever, we execute a merge query, a node is either matched or created. Using on create and on match, you can set properties for indicating whether the node is created or matched.

Following is a sample Cypher Query which demonstrates the usage of **OnCreate** and **OnMatch** clauses in Neo4j. If the specified node already exists in the database, then the node will be matched and the property with key-value pair `isFound = "true"` will be created in the node

If the specified node doesn't exist in the database, then the node will be created, and within it a property with a key-value pair `isCreated = "true"` will be created.


```
MERGE (afridi:player {name: "shaheen afridi", YOB: 1998, POB: "peshwar"}) ON CREATE
SET afridi.isCreated = "true" ON MATCH SET afridi.isFound = "true" RETURN afridi
```

Merge a Relationship: Just like nodes, you can also merge the relationships using the MERGE clause. Following is a sample Cypher Query which merges a relationship using the MATCH clause in Neo4j. This query tries to merge a relationship named **WINNERS_OF** between the nodes PAK (label: Country & name: pakistan) and ICC13 (label: Tournament & name: ICC Champions Trophy 2013).

Example,

```
MATCH (a:Country), (b:Tournament)
  WHERE a.name = "pakistan" AND b.name = "ICC Champions Trophy 2013"
  MERGE (a)-[r:WINNERS_OF]->(b)
RETURN a, b
```

Neo4j CQL – Set Clause

Using Set clause, you can add new properties to an existing Node or Relationship, and also add or update existing Properties values.

Syntax,

```
MATCH (node:label{properties . . . . . })
SET node.property = value
RETURN node
```

```
CREATE (babar:player{name: "babar azam", YOB: 1998, POB: "lahore"})
```

```
MATCH (babar:player{name: "babar azam", YOB: 1998, POB: "lahore"})
SET babar.highestscore = 187
RETURN babar
```

Remove a property: You can remove an existing property by passing **NULL** as value to it.

Syntax,

```
MATCH (node:label {properties})
SET node.property = NULL
RETURN node
```

Example,

```
MATCH (babar:player {name: "babar azam", YOB: 1998, POB: "lahore"})
SET babar.POB = NULL
RETURN babar
```

Setting multiple property: In the same way, you can create multiple properties in a node using the Set clause. To do so, you need to specify these key value pairs with commas

Syntax,

```
MATCH (node:label {properties})
SET node.property1 = value, node.property2 = value
RETURN node
```

Setting a Label on n Node: You can set a label to an existing node using the SET clause.

Syntax,

```
MATCH (n {properties . . . . . })
SET n :label
RETURN n
```

Example,

CREATE (Anderson {**name**: "James Anderson", YOB: 1982, POB: "Burnely"})

MATCH (Anderson {**name**: "James Anderson", YOB: 1982, POB: "Burnely"})

SET Anderson: player

RETURN Anderson

Neo4j CQL – Delete Clause

Deleting all nodes and relationships: Following is the query to delete all the nodes and the relationships in the database using the DELETE clause.

Syntax,

```
MATCH (n) DETACH DELETE n
```

Delete a particular node: To delete a particular node, you need to specify the details of the node in the place of n in the above query

Syntax,

```
MATCH (node:label {properties . . . . . })
```

```
DETACH DELETE node
```

Example,

```
CREATE (rizwan:player {name: "muhammad rizwan", YOB: 1999, POB: "mardan"})
```

```
MATCH (rizwan:player {name: "muhammad rizwan", YOB: 1999, POB: "mardan"})  
DETACH DELETE rizwan
```

Neo4j CQL – For Each

```
CREATE p = (rizwan {name:"muhammad rizwan"})-[:TOPSCORRER_OF]->(pak{name:
    "pakistan"})-[:WINNER_OF]->(CT2013{name: "Champions Trophy 2013"})
RETURN p
```

```
MATCH p = (rizwan)-[*]->(CT2013)
    WHERE rizwan.name = "muhammad rizwan" AND CT2013.name = "Champions Trophy 201
3"
FOREACH (n IN nodes(p) | SET n.marked = TRUE)
```

Neo4j CQL – Match Clause

```
CREATE (babar:player {name: "babar azam", YOB: 1981, POB: "lahore"})
CREATE (pak:Country {name: "pakistan", result: "Winners"})
CREATE (CT2013:Tournament {name: "ICC Champions Trophy 2013"})
CREATE (Ind)-[r1:WINNERS_OF {NRR:0.938 ,pts:6}]->(CT2013)

CREATE (babar)-[r2:CAPTAIN_OF]->(pak)
CREATE (rizwan:player {name: "Muhammad rizwan", YOB: 1995, POB: "peshawar"})
CREATE (shaheen:player {name: "shaheen Afridi", YOB: 1988, POB: "peshawar"})

CREATE (rizwan)-[:TOP_SCORER_OF {Runs:363}]->(pak)
CREATE (shaheen)-[:HIGHEST_WICKET_TAKER_OF {Wickets:12}]->(pak)
```